

SVM & NAIVE BAYES

Complete Assignment Answers

PART A – THEORETICAL QUESTIONS

Question 1: What is a Support Vector Machine (SVM)?

Answer:

Support Vector Machine (SVM) is a supervised Machine Learning algorithm used mainly for **classification** and also for **regression**.

SVM finds the best decision boundary, called a **hyperplane**, that separates different classes of data.

The main objective of SVM is to find a hyperplane that provides the **maximum margin** between the classes.

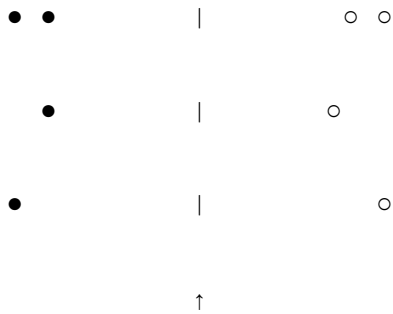
Key Terms:

- **Hyperplane:** Decision boundary that separates classes.
- **Margin:** Distance between the hyperplane and the nearest data points.
- **Support Vectors:** Data points closest to the decision boundary.

Example:

For two classes:

Class A | Class B



Decision Boundary

Easy Definition:

SVM finds the best boundary that separates classes with the maximum possible margin.

Question 2: What is the Difference Between Hard Margin and Soft Margin SVM?

Answer:

Hard Margin and Soft Margin are two approaches used by SVM to handle classification data.

Hard Margin SVM

Hard Margin SVM requires **all training points to be correctly classified**.

It does not allow points to cross the margin or be misclassified.

Advantages:

- Works well when classes are perfectly separable.
- Produces a strict decision boundary.

Disadvantage:

It is highly sensitive to noise and outliers.

Soft Margin SVM

Soft Margin SVM allows some observations to violate the margin or even be misclassified.

It uses the **C parameter** to control the trade-off between:

- Large margin
- Classification errors

Comparison:

Hard Margin	Soft Margin
No classification errors allowed	Some errors allowed
Suitable for perfectly separable data	Suitable for noisy real-world data
Sensitive to outliers	More robust to noise
Strict boundary	Flexible boundary

Easy Definition:

Hard Margin allows no mistakes, while Soft Margin allows some mistakes to obtain a better generalizing boundary.

Question 3: What is the Mathematical Intuition Behind SVM?

Answer:

SVM tries to find a hyperplane that separates different classes while **maximizing the margin**.

For a linear SVM, the decision boundary can be written as:

$$w^T x + b = 0$$

Where:

- w = weight vector
- x = input feature vector
- b = bias/intercept

The distance of a point from the hyperplane depends on:

$$\frac{|w^T x + b|}{\|w\|}$$

SVM maximizes this distance for the nearest training points.

For a normalized formulation, the margin is related to:

$$\frac{2}{\|w\|}$$

Therefore, maximizing the margin is equivalent to minimizing:

$$\frac{1}{2} \|w\|_2^2$$

subject to classification constraints.

Easy Definition:

SVM chooses the boundary that separates classes while keeping the nearest points as far away as possible.

Question 4: What is the Role of Lagrange Multipliers in SVM?

Answer:

Lagrange Multipliers are used to solve the constrained optimization problem in SVM.

The original SVM problem contains constraints such as:

$$y_i (w^T x_i) \geq 1$$

$$\alpha_i (w \cdot x_i + b) \geq 1$$

Lagrange multipliers, usually represented by (α_i) , allow these constraints to be incorporated into the optimization objective.

The resulting dual formulation helps determine which training observations are important.

Important Point:

For many observations:

$$\alpha_i = 0$$

For support vectors:

$$\alpha_i > 0$$

Therefore, Lagrange multipliers help identify the **Support Vectors**.

Easy Definition:

Lagrange Multipliers help SVM solve its constrained optimization problem and identify the important support vectors.

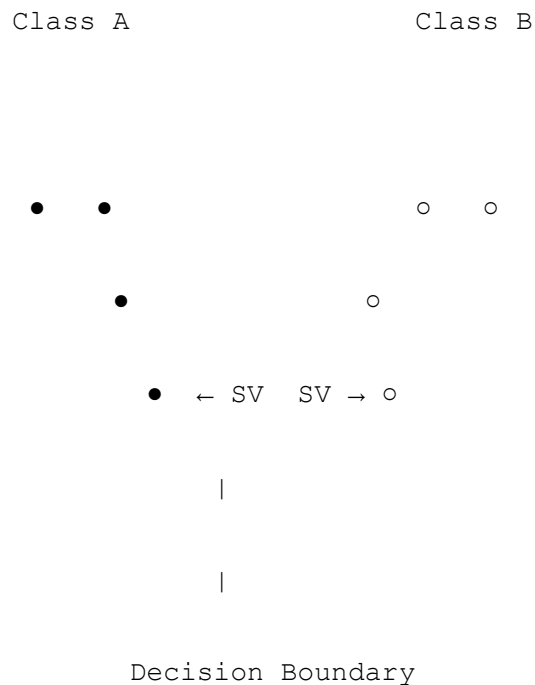
Question 5: What are Support Vectors in SVM?

Answer:

Support Vectors are the training data points that are closest to the decision boundary or that influence the optimal margin.

They are extremely important because the position of the decision boundary depends mainly on these points.

Example:



Removing or changing a support vector can change the decision boundary.

Easy Definition:

Support vectors are the critical data points that determine the SVM decision boundary.

Question 6: What is a Support Vector Classifier (SVC)?

Answer:

Support Vector Classifier (SVC) is an SVM-based algorithm used for classification.

In Scikit-learn, it is implemented using:

```
from sklearn.svm import SVC
```

Example:

```
model = SVC(kernel="linear")
```

SVC can use different kernels such as:

- Linear
- Polynomial
- RBF
- Sigmoid

It can also handle multi-class classification.

Easy Definition:

SVC is the SVM algorithm used to classify data into different categories.

Question 7: What is a Support Vector Regressor (SVR)?

Answer:

Support Vector Regressor (SVR) is the regression version of SVM.

Instead of separating classes, SVR tries to fit a function within a specified **epsilon-insensitive tube** around the target values.

Points inside the epsilon tube do not contribute to the loss.

Basic Idea:

Upper ε boundary

• •

•

Regression Line

•

• •

Lower ε boundary

Python:

```
from sklearn.svm import SVR
```

```
model = SVR(kernel="rbf")
```

Easy Definition:

SVR predicts continuous values while trying to keep prediction errors within an epsilon margin.

Question 8: What is the Kernel Trick in SVM?

Answer:

The **Kernel Trick** allows SVM to create non-linear decision boundaries without explicitly transforming the data into a high-dimensional feature space.

A kernel calculates similarity between observations as if the observations had been mapped into another feature space.

Common kernels include:

- Linear
- Polynomial
- RBF
- Sigmoid

Example:

If the original data cannot be separated using a straight line, an RBF kernel can create a non-linear boundary.

Easy Definition:

The Kernel Trick helps SVM solve non-linear problems efficiently by computing similarities in a transformed feature space.

Question 9: Compare Linear Kernel, Polynomial Kernel, and RBF Kernel.

Answer:

Kernel	Description	Suitable For
Linear	Creates a linear decision boundary	Linearly separable data
Polynomial	Creates polynomial/non-linear boundaries	Polynomial relationships
RBF	Creates flexible non-linear boundaries	Complex non-linear data

Linear Kernel

```
SVC(kernel="linear")
```

Polynomial Kernel

```
SVC(kernel="poly", degree=3)
```

RBF Kernel

```
SVC(kernel="rbf")
```

RBF Formula:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Easy Definition:

Linear is simple, Polynomial models polynomial relationships, and RBF is highly flexible for complex non-linear patterns.

Question 10: What is the Effect of the C Parameter in SVM?

Answer:

The **C parameter** controls the trade-off between maximizing the margin and minimizing classification errors.

Small C:

- Allows more classification errors.
- Produces a wider margin.
- Can provide better generalization.
- May underfit if too small.

Large C:

- Penalizes errors heavily.
- Tries to classify training observations correctly.
- Produces a narrower margin.
- Can overfit noisy data.

Summary:

C Value	Behavior
Small C	Wider margin, more errors allowed
Large C	Narrower margin, fewer training errors

Easy Definition:

C controls how strongly SVM penalizes classification errors.

Question 11: What is the Role of the Gamma Parameter in RBF Kernel SVM?

Answer:

Gamma (γ) controls how far the influence of an individual training point extends.

Low Gamma:

- Each point has a broader influence.
- Decision boundary is smoother.
- Model may underfit.

High Gamma:

- Each point has a smaller influence.
- Decision boundary becomes more complex.
- Model may overfit.

Summary:

Gamma	Model Behavior
Low	Smooth/simple boundary
High	Complex boundary

Easy Definition:

Gamma controls the influence range of individual data points in an RBF SVM.

Question 12: What is the Naïve Bayes Classifier, and Why is it Called "Naïve"?

Answer:

Naïve Bayes is a supervised Machine Learning classification algorithm based on **Bayes' Theorem**.

It is called "**Naïve**" because it assumes that the features are conditionally independent given the class.

For example, in text classification, it may treat the presence of individual words as conditionally independent given the class, even though words can actually be related.

Formula:

$$P(C|X) \propto P(C)P(X|C)$$

With the naïve independence assumption:

$$P(C|X) \propto P(C) \prod_i P(x_i|C)$$

Advantages:

- Fast
- Simple
- Effective for many classification problems
- Works especially well with text data

Easy Definition:

Naïve Bayes is a probability-based classifier that assumes features are conditionally independent.

Question 13: What is Bayes' Theorem?

Answer:

Bayes' Theorem calculates the probability of an event based on prior knowledge and new evidence.

Formula:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where:

- $P(A|B)$ = Posterior Probability
- $P(B|A)$ = Likelihood
- $P(A)$ = Prior Probability
- $P(B)$ = Evidence

Example:

Suppose a medical test is positive.

Bayes' theorem can help calculate:

Probability that the patient actually has the disease given that the test is positive.

Easy Definition:

Bayes' theorem updates our belief about an event after observing new evidence.

Question 14: Explain the Differences Between Gaussian, Multinomial, and Bernoulli Naïve Bayes.

Answer:

There are several variants of Naïve Bayes.

1. Gaussian Naïve Bayes

Used when features are continuous numerical values and are commonly modeled using a Gaussian distribution within each class.

Example:

- Age
- Height
- Weight
- Blood pressure

Python:

```
from sklearn.naive_bayes import GaussianNB
```

2. Multinomial Naïve Bayes

Commonly used for discrete counts or frequencies.

It is particularly popular for text classification.

Example:

- Word counts
- Document-term frequencies

Python:

```
from sklearn.naive_bayes import MultinomialNB
```

3. Bernoulli Naïve Bayes

Used when features are binary.

Example:

```
Word present = 1
```

```
Word absent = 0
```

Python:

```
from sklearn.naive_bayes import BernoulliNB
```

Comparison:

Type

Data

GaussianNB	Continuous numerical features
------------	-------------------------------

MultinomialNB	Counts/frequencies
---------------	--------------------

BernoulliNB	Binary features
-------------	-----------------

Question 15: When Should You Use Gaussian Naïve Bayes Over Other Variants?

Answer:

Gaussian Naïve Bayes should generally be used when the input features are **continuous numerical variables** and the Gaussian assumption is reasonably appropriate.

Examples:

- Height
- Weight
- Temperature
- Blood pressure
- Age
- Measurements

Example:

```
from sklearn.naive_bayes import GaussianNB
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

Easy Definition:

Use GaussianNB mainly when your features are continuous numerical measurements.

Question 16: What are the Key Assumptions Made by Naïve Bayes?

Answer:

The most important assumption is:

Conditional Independence

Features are assumed to be conditionally independent given the class.

For example:

Feature 1 $\neg$

Feature 2 $\perp \rightarrow$ Class

Feature 3 $\perp$

Naïve Bayes also relies on distributional assumptions depending on its variant:

- GaussianNB $\rightarrow$ Gaussian distribution assumption

- MultinomialNB → count/frequency-based model
- BernoulliNB → binary features

Easy Definition:

The main Naïve Bayes assumption is that features are conditionally independent given the class.

Question 17: What are the Advantages and Disadvantages of Naïve Bayes?

Advantages:

1. Very fast to train.
2. Fast prediction.
3. Works well with high-dimensional data.
4. Requires relatively little training data.
5. Performs well for many text classification problems.
6. Simple to implement.

Disadvantages:

1. Conditional independence assumption may be unrealistic.
2. Performance can suffer when features are strongly dependent.
3. Zero-frequency problems can occur without smoothing.
4. Probability estimates may not always be well calibrated.

Comparison:

Advantages	Disadvantages
Fast	Independence assumption
Simple	Correlated features can be problematic

Good for text

Zero-frequency problem

Works with high-dimensional data

Probability estimates may need calibration

Question 18: Why is Naïve Bayes a Good Choice for Text Classification?

Answer:

Naïve Bayes is widely used for text classification because text datasets usually contain a **large number of features**, such as words or tokens.

It is:

- Fast
- Computationally efficient
- Effective with sparse data
- Suitable for high-dimensional feature spaces

Applications:

- Spam detection
- Sentiment analysis
- News classification
- Topic classification
- Document categorization

Example:

Email:

"Congratulations! You won a prize."

Naïve Bayes:

Spam → High Probability

Easy Definition:

Naïve Bayes is fast and effective for high-dimensional text data, making it useful for spam and document classification.

Question 19: Compare SVM and Naïve Bayes for Classification Tasks.

Answer:

SVM	Naïve Bayes
Margin-based algorithm	Probability-based algorithm
Usually needs feature scaling for many kernels	Often works without scaling
Can model complex boundaries using kernels	Uses probabilistic assumptions
Can be computationally expensive for large datasets	Very fast
Excellent for many classification problems	Excellent for many text problems
Less dependent on probability assumptions	Assumes conditional independence
Hyperparameter tuning may be important	Usually simple to tune

When to Use SVM:

Use SVM when:

- Dataset is small/medium-sized.
- Decision boundary is complex.
- High predictive performance is required.

When to Use Naïve Bayes:

Use Naïve Bayes when:

- Dataset is large and high-dimensional.
 - Text classification is involved.
 - Fast training is important.
-

Question 20: How Does Laplace Smoothing Help in Naïve Bayes?

Answer:

Laplace Smoothing prevents the probability of an unseen feature/class combination from becoming zero.

Without smoothing, if a word has never appeared in a particular class:

$$P(\text{word}|\text{class})=0$$

Because Naïve Bayes multiplies probabilities, one zero probability can make the entire class probability zero.

Laplace Formula:

$$P($$

$$P(x_i | C) = \frac{\text{count}(x_i, C) + \alpha}{\text{count}(C) + \alpha K}$$

Where:

- α = smoothing parameter
- K = number of possible feature values/categories

Commonly:

$$\alpha = 1$$

Easy Definition:

Laplace Smoothing adds a small value to counts so that unseen events do not receive zero probability.

PART B – PRACTICAL QUESTIONS

Question 21: Train an SVM Classifier on the Iris Dataset and Evaluate Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


# Load Iris dataset

iris = load_iris()


X = iris.data

y = iris.target


# Split data

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,

    stratify=y

)
```

```
# Feature scaling

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# Train SVM

model = SVC(kernel="rbf", C=1, gamma="scale")

model.fit(X_train_scaled, y_train)


# Prediction

y_pred = model.predict(X_test_scaled)


# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("SVM Accuracy:", accuracy)
```

Observation:

Feature scaling is important for SVM because the algorithm is sensitive to the scale of the input features.

Question 22: Train Linear and RBF SVM Classifiers on the Wine Dataset and Compare Accuracies.

Python Code:

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


# Load Wine dataset

wine = load_wine()


X = wine.data
```

```
y = wine.target

# Split data

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,

    stratify=y

)

# Scale features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# Linear SVM

linear_svm = SVC(kernel="linear")
```

```
linear_svm.fit(X_train_scaled, y_train)
```

```
linear_pred = linear_svm.predict(X_test_scaled)
```

```
linear_accuracy = accuracy_score(
```

```
    y_test,
```

```
    linear_pred
```

```
)
```

```
# RBF SVM
```

```
rbf_svm = SVC(kernel="rbf")
```

```
rbf_svm.fit(X_train_scaled, y_train)
```

```
rbf_pred = rbf_svm.predict(X_test_scaled)
```

```
rbf_accuracy = accuracy_score(
```

```
    y_test,
```

```
    rbf_pred
```

```
)
```

```
print("Linear Kernel Accuracy:", linear_accuracy)
```

```
print("RBF Kernel Accuracy:", rbf_accuracy)
```

Observation:

The better kernel is the one with higher test/validation performance. The RBF kernel can model non-linear boundaries, while the linear kernel is simpler.

Question 23: Train an SVM Regressor (SVR) on a Housing Dataset and Evaluate Using MSE.

Answer:

The old Boston Housing dataset is no longer included in modern Scikit-learn versions. A convenient modern alternative is the **California Housing dataset**.

Python Code:

```
from sklearn.datasets import fetch_california_housing
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVR
```

```
from sklearn.metrics import mean_squared_error

# Load California Housing dataset

housing = fetch_california_housing()

X = housing.data

y = housing.target

# Split data

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42

)

# Scale features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
```



```
X_test_scaled = scaler.transform(X_test)
```

```
# Create SVR model
```

```
model = SVR(
```

```
    kernel="rbf",
```

```
    C=100,
```

```
    epsilon=0.1
```

```
)
```

```
# Train model
```

```
model.fit(X_train_scaled, y_train)
```

```
# Predictions
```

```
y_pred = model.predict(X_test_scaled)
```

```
# MSE
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error:", mse)
```

Observation:

Lower MSE indicates smaller squared prediction errors.

Question 24: Train an SVM Classifier with a Polynomial Kernel and Visualize the Decision Boundary.

Python Code:

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC


# Create a 2D dataset

X, y = make_classification(
```

```
n_samples=300,

n_features=2,

n_redundant=0,

n_informative=2,

n_clusters_per_class=1,

class_sep=1.5,

random_state=42

)


# Scale features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


# Polynomial SVM

model = SVC(

    kernel="poly",

    degree=3,

    C=1

)
```

```
model.fit(X_scaled, y)
```

```
# Create mesh
```

```
x_min = X_scaled[:, 0].min() - 1
```

```
x_max = X_scaled[:, 0].max() + 1
```

```
y_min = X_scaled[:, 1].min() - 1
```

```
y_max = X_scaled[:, 1].max() + 1
```

```
xx, yy = np.meshgrid(  
    np.linspace(x_min, x_max, 500),  
    np.linspace(y_min, y_max, 500)  
)
```

```
Z = model.predict(  
    np.c_[xx.ravel(), yy.ravel()]  
)
```

```
Z = Z.reshape(xx.shape)
```

```
# Plot decision boundary

plt.figure(figsize=(8, 6))

plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(

    X_scaled[:, 0],

    X_scaled[:, 1],

    c=y,

    edgecolors="k"

)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.title("Polynomial Kernel SVM Decision Boundary")

plt.show()
```

Observation:

The polynomial kernel can create non-linear decision boundaries.

Question 25: Train a Gaussian Naïve Bayes Classifier on the Breast Cancer Dataset and Evaluate Accuracy.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


# Load dataset

data = load_breast_cancer()


X = data.data

y = data.target


# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(  
  
    X, y,  
  
    test_size=0.2,  
  
    random_state=42,  
  
    stratify=y  
  
)
```

```
# Create Gaussian Naive Bayes
```

```
model = GaussianNB()
```

```
# Train
```

```
model.fit(X_train, y_train)
```

```
# Predict
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Gaussian Naive Bayes Accuracy:", accuracy)
```

Observation:

GaussianNB is appropriate here because the dataset contains continuous numerical measurements.

Question 26: Train a Multinomial Naïve Bayes Classifier for Text Classification Using the 20 Newsgroups Dataset.

Python Code:

```
from sklearn.datasets import fetch_20newsgroups

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.naive_bayes import MultinomialNB

from sklearn.pipeline import Pipeline

from sklearn.metrics import accuracy_score

# Load training data

train_data = fetch_20newsgroups(
```



```
subset="train",

remove=("headers", "footers", "quotes")

)
```

```
# Load test data
```

```
test_data = fetch_20newsgroups(

    subset="test",

    remove=("headers", "footers", "quotes")

)
```

```
# Create pipeline
```

```
model = Pipeline([

    ("tfidf", TfidfVectorizer(

        stop_words="english"

    )),

    ("classifier", MultinomialNB())

])
```

```
# Train
```

```
model.fit(  
  
    train_data.data,  
  
    train_data.target  
  
)  
  
# Predict  
  
predictions = model.predict(test_data.data)  
  
# Accuracy  
  
accuracy = accuracy_score(  
  
    test_data.target,  
  
    predictions  
  
)  
  
print("Accuracy:", accuracy)
```

Observation:

Multinomial Naïve Bayes is widely used for text classification because it works efficiently with word-frequency or TF-IDF-style non-negative features.

Question 27: Train an SVM Classifier with Different C Values and Compare Decision Boundaries Visually.

Python Code:

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import make_moons

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC


# Create dataset

X, y = make_moons(

    n_samples=300,

    noise=0.2,

    random_state=42

)
```

```
# Scale data

scaler = StandardScaler()

X = scaler.fit_transform(X)

C_values = [0.1, 1, 10, 100]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for ax, C in zip(axes.ravel(), C_values):

    model = SVC(

        kernel="rbf",

        C=C,

        gamma="scale"

    )

    model.fit(X, y)

    x_min = X[:, 0].min() - 1
```

```
x_max = X[:, 0].max() + 1

y_min = X[:, 1].min() - 1

y_max = X[:, 1].max() + 1


xx, yy = np.meshgrid(

    np.linspace(x_min, x_max, 300),

    np.linspace(y_min, y_max, 300)

)


Z = model.predict(

    np.c_[xx.ravel(), yy.ravel()]

)


Z = Z.reshape(xx.shape)


ax.contourf(xx, yy, Z, alpha=0.3)


ax.scatter(

    X[:, 0],
```

```
X[:, 1],

c=y,

edgecolors="k"

)

ax.set_title(f"SVM with C = {C}")

plt.tight_layout()

plt.show()
```

Observation:

- Small C → more tolerance for errors and smoother boundary.
 - Large C → stronger penalty for errors and potentially more complex boundary.
-

Question 28: Train a Bernoulli Naïve Bayes Classifier for Binary Classification.

Python Code:

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import BernoulliNB
```

```
from sklearn.metrics import accuracy_score
```

```
# Create binary features
```

```
X = np.array([
```

```
    [1, 0, 1, 0],
```

```
    [1, 1, 1, 0],
```

```
    [0, 1, 0, 1],
```

```
    [0, 1, 0, 1],
```

```
    [1, 0, 1, 1],
```

```
    [0, 0, 0, 1],
```

```
    [1, 1, 0, 0],
```

```
    [0, 1, 1, 1],
```

```
    [1, 0, 0, 0],
```

```
    [0, 0, 1, 1]
```

```
])
```

```
y = np.array([
```

```

    1, 1, 0, 0, 1,

    0, 1, 0, 1, 0

])

# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.3,

    random_state=42,

    stratify=y

)

# Bernoulli Naive Bayes

model = BernoulliNB()

model.fit(X_train, y_train)

# Prediction

y_pred = model.predict(X_test)

```



```
# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Observation:

BernoulliNB is suitable when features represent binary events such as **present/absent** or **0/1**.

Question 29: Apply Feature Scaling Before Training an SVM and Compare Results with Unscaled Data.

Python Code:

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_wine()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# SVM without scaling
```

```
model_unscaled = SVC(kernel="rbf")
```

```
model_unscaled.fit(X_train, y_train)
```

```
pred_unscaled = model_unscaled.predict(X_test)
```

```
accuracy_unscaled = accuracy_score(
```

```
    y_test,
```

```
    pred_unscaled
```

```
)
```

```
# Scaling
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# SVM with scaling
```

```
model_scaled = SVC(kernel="rbf")
```

```
model_scaled.fit(
```

```
    X_train_scaled,
```

```
        y_train
    )

    pred_scaled = model_scaled.predict(

        X_test_scaled
    )

    accuracy_scaled = accuracy_score(

        y_test,

        pred_scaled
    )

    print("Accuracy Without Scaling:",

        accuracy_unscaled)

    print("Accuracy With Scaling:",

        accuracy_scaled)
```

Observation:

SVM, especially with RBF and polynomial kernels, is sensitive to feature scale. Scaling often improves or stabilizes performance.

Question 30: Train a Gaussian Naïve Bayes Model and Compare Predictions Before and After Laplace Smoothing.

Important Concept:

The wording of this question needs an important clarification.

Laplace smoothing is naturally used with discrete count-based Naïve Bayes models such as MultinomialNB and BernoulliNB. GaussianNB models continuous variables using Gaussian distributions and does not use the standard Laplace smoothing parameter.

Therefore, we can demonstrate the intended smoothing concept using **MultinomialNB**.

Python Code:

```
import numpy as np

from sklearn.naive_bayes import MultinomialNB

# Simple count-based dataset

X_train = np.array([
```

```
[3, 0, 1],

[2, 1, 0],

[0, 3, 2],

[0, 2, 3]

])

y_train = np.array([

    0, 0, 1, 1

])

X_test = np.array([

    [0, 0, 1],

    [1, 0, 0]

])

# Without smoothing

model_no_smoothing = MultinomialNB(alpha=0)

model_no_smoothing.fit(
```

```
        X_train,

        y_train

    )

    pred_no_smoothing = model_no_smoothing.predict(

        X_test

    )
```

```
# With Laplace smoothing
```

```
model_smoothing = MultinomialNB(alpha=1)
```

```
model_smoothing.fit(

    X_train,

    y_train

)

    pred_smoothing = model_smoothing.predict(

        X_test

    )
```

```
print("Predictions without smoothing:",  
      pred_no_smoothing)  
  
print("Predictions with Laplace smoothing:",  
      pred_smoothing)
```

Observation:

Laplace smoothing prevents zero probabilities for unseen feature events.

Question 31: Train an SVM Classifier and Use GridSearchCV to Tune C, Gamma, and Kernel.

Python Code:

```
from sklearn.datasets import load_iris  
  
from sklearn.model_selection import (  
    train_test_split,  
    GridSearchCV  
)
```



```
from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import Pipeline

from sklearn.svm import SVC


# Load dataset

data = load_iris()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,

    stratify=y

)


# Pipeline
```

```
pipeline = Pipeline([

    ("scaler", StandardScaler()),

    ("svc", SVC())

])

# Parameters

param_grid = [

    {

        "svc__kernel": ["linear"],

        "svc__C": [0.1, 1, 10, 100]

    },

    {

        "svc__kernel": ["rbf"],

        "svc__C": [0.1, 1, 10, 100],

        "svc__gamma": ["scale", 0.01, 0.1, 1]

    },

    {

        "svc__kernel": ["poly"],

        "svc__C": [0.1, 1, 10],
```

```
        "svc__gamma": ["scale", 0.01, 0.1],

        "svc__degree": [2, 3]

    }

]
```

```
# Grid Search
```

```
grid = GridSearchCV(

    pipeline,

    param_grid,

    cv=5,

    scoring="accuracy"

)
```

```
grid.fit(X_train, y_train)
```

```
print("Best Parameters:")
```

```
print(grid.best_params_)
```

```
print("\nBest CV Accuracy:")
```

```
print(grid.best_score_)

print("\nTest Accuracy:")

print(grid.score(X_test, y_test))
```

Question 32: Train an SVM on an Imbalanced Dataset and Apply Class Weighting.

Python Code:

```
from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score

# Create imbalanced dataset

X, y = make_classification(

    n_samples=1000,
```

```
n_features=10,

n_informative=5,

weights=[0.90, 0.10],

random_state=42

)


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,

    stratify=y

)


# Scale

scaler = StandardScaler()


X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

```
# Without class weighting
```

```
model_normal = SVC(kernel="rbf")
```

```
model_normal.fit(  
  
    X_train_scaled,  
  
    y_train  
  
)
```

```
pred_normal = model_normal.predict(  
  
    X_test_scaled  
  
)
```

```
accuracy_normal = accuracy_score(  
  
    y_test,  
  
    pred_normal  
  
)
```

```
# With class weighting
```

```
model_weighted = SVC(
```

```
    kernel="rbf",
```

```
    class_weight="balanced"
```

```
)
```

```
model_weighted.fit(
```

```
    X_train_scaled,
```

```
    y_train
```

```
)
```

```
pred_weighted = model_weighted.predict(
```

```
    X_test_scaled
```

```
)
```

```
accuracy_weighted = accuracy_score(
```

```
    y_test,
```

```
    pred_weighted
```

```
)
```

```
print("Accuracy Without Class Weighting:",  
      accuracy_normal)
```

```
print("Accuracy With Class Weighting:",  
      accuracy_weighted)
```

Important Observation:

Accuracy alone can be misleading for imbalanced datasets. Also compare:

- Precision
- Recall
- F1-score
- Confusion Matrix
- ROC-AUC / PR-AUC

Class weighting does not necessarily improve raw accuracy; its purpose is to give more importance to the minority class.

Question 33: Implement a Naïve Bayes Classifier for Spam Detection Using Email Data.

Python Code:

```
from sklearn.model_selection import train_test_split  
  
from sklearn.feature_extraction.text import TfidfVectorizer
```



```
from sklearn.naive_bayes import MultinomialNB

from sklearn.pipeline import Pipeline

from sklearn.metrics import accuracy_score


# Example email dataset

emails = [

    "Congratulations you won a free prize",

    "Claim your free reward now",

    "Win money by clicking this link",

    "Meeting is scheduled for tomorrow",

    "Please submit your project report",

    "Let's discuss the project in the meeting",

    "You have won a lottery prize",

    "Free gift waiting for you"

]


# 1 = Spam, 0 = Not Spam

labels = [

    1, 1, 1, 0,
```

```
0, 0, 1, 1

]

# Split data

X_train, X_test, y_train, y_test = train_test_split(

    emails,

    labels,

    test_size=0.25,

    random_state=42,

    stratify=labels

)

# Pipeline

model = Pipeline([

    ("tfidf", TfidfVectorizer()),

    ("nb", MultinomialNB())

])

# Train
```

```
model.fit(X_train, y_train)
```

```
# Predict
```

```
predictions = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(
```

```
    y_test,
```

```
    predictions
```

```
)
```

```
print("Spam Detection Accuracy:", accuracy)
```

```
# Test new email
```

```
new_email = [
```

```
    "You won a free cash prize"
```

```
]
```

```
prediction = model.predict(new_email)
```

```
print("Prediction:", prediction)
```

Note:

For a real project, use a sufficiently large labeled email dataset instead of the small illustrative dataset above.

Question 34: Train SVM and Naïve Bayes on the Same Dataset and Compare Accuracy.

Python Code:

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


# Load dataset
```

```
data = load_wine()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Scaling for SVM
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# SVM
```

```
svm_model = SVC(kernel="rbf")
```

```
svm_model.fit(
```

```
    X_train_scaled,
```

```
    y_train
```

```
)
```

```
svm_pred = svm_model.predict(
```

```
    X_test_scaled
```

```
)
```

```
svm_accuracy = accuracy_score(
```

```
    y_test,
```

```
    svm_pred
```

```
)
```

```
# Gaussian Naive Bayes
```

```
nb_model = GaussianNB()
```

```
nb_model.fit(  
  
    X_train,  
  
    y_train  
  
)  
  
nb_pred = nb_model.predict(X_test)  
  
nb_accuracy = accuracy_score(  
  
    y_test,  
  
    nb_pred  
  
)  
  
print("SVM Accuracy:", svm_accuracy)  
  
print("Naive Bayes Accuracy:", nb_accuracy)
```

Observation:

The algorithm with higher validation/test performance is preferable for this particular dataset, but model selection should also consider speed, interpretability, calibration, and the business objective.

Question 35: Perform Feature Selection Before Training a Naïve Bayes Classifier and Compare Results.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.feature_selection import SelectKBest, f_classif

from sklearn.pipeline import Pipeline

from sklearn.metrics import accuracy_score


# Load dataset

data = load_breast_cancer()


X = data.data
```



```
y = data.target

# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,

    stratify=y

)

# Model without feature selection

model_all = GaussianNB()

model_all.fit(

    X_train,

    y_train

)

pred_all = model_all.predict(X_test)
```

```
accuracy_all = accuracy_score(  
  
    y_test,  
  
    pred_all  
  
)
```

```
# Feature selection
```

```
pipeline = Pipeline([  
  
    ("selector", SelectKBest(  
  
        score_func=f_classif,  
  
        k=10  
  
    )),  
  
    ("nb", GaussianNB())  
  
])
```

```
pipeline.fit(  
  
    X_train,  
  
    y_train  
  
)
```

```
pred_selected = pipeline.predict(X_test)
```

```
accuracy_selected = accuracy_score(
```

```
    y_test,
```

```
    pred_selected
```

```
)
```

```
print("Accuracy Using All Features:",
```

```
      accuracy_all)
```

```
print("Accuracy Using Selected Features:",
```

```
      accuracy_selected)
```

Observation:

Feature selection may improve, maintain, or reduce accuracy. Its main benefit can also be reducing dimensionality and computational complexity.

Question 36: Train an SVM Using One-vs-Rest and One-vs-One Strategies on the Wine Dataset.

Python Code:

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.multiclass import OneVsRestClassifier

from sklearn.metrics import accuracy_score


# Load dataset

data = load_wine()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y,

test_size=0.2,

random_state=42,

stratify=y

)


# Scale

scaler = StandardScaler()


X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# One-vs-Rest

ovr_model = OneVsRestClassifier(

    SVC(kernel="linear")

)


ovr_model.fit(

    X_train_scaled,
```

```
        y_train

    )

    ovr_pred = ovr_model.predict(

        X_test_scaled

    )

    ovr_accuracy = accuracy_score(

        y_test,

        ovr_pred

    )

# One-vs-One

ovo_model = SVC(

    kernel="linear",

    decision_function_shape="ovo"

)

ovo_model.fit(
```

```
        X_train_scaled,

        y_train

    )


    ovo_pred = ovo_model.predict(

        X_test_scaled

    )


    ovo_accuracy = accuracy_score(

        y_test,

        ovo_pred

    )


    print("One-vs-Rest Accuracy:", ovr_accuracy)

    print("One-vs-One Accuracy:", ovo_accuracy)
```

Concept:

For (K) classes:

OvR: Builds approximately K binary classifiers.

OvO: Builds one classifier for every pair of classes:

$$\frac{K(K-1)}{2}$$

Question 37: Train SVM with Linear, Polynomial, and RBF Kernels on Breast Cancer Dataset.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


# Load dataset

data = load_breast_cancer()


X = data.data

y = data.target
```



```
# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,

    stratify=y

)
```

```
# Scale
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
kernels = [
```

```
    ("Linear", SVC(kernel="linear")),
```

```
    ("Polynomial", SVC(kernel="poly", degree=3)),
```

```
    ("RBF", SVC(kernel="rbf"))
```

```
]
```

```
for name, model in kernels:

    model.fit(

        X_train_scaled,

        y_train

    )

    predictions = model.predict(

        X_test_scaled

    )

    accuracy = accuracy_score(

        y_test,

        predictions

    )

    print(name, "Kernel Accuracy:", accuracy)
```

Observation:

The best-performing kernel depends on the dataset and hyperparameters. RBF is often a strong baseline for non-linear patterns.

Question 38: Train an SVM Using Stratified K-Fold Cross-Validation and Compute Average Accuracy.

Python Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import (

    StratifiedKFold,

    cross_val_score

)

from sklearn.pipeline import Pipeline

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

import numpy as np

# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Pipeline
```

```
model = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm", SVC(kernel="rbf"))  
])
```

```
# Stratified K-Fold
```

```
skf = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

```
# Cross-validation
```

```
scores = cross_val_score(  
  
    model,  
  
    X,  
  
    y,  
  
    cv=skf,  
  
    scoring="accuracy"  
  
)  
  
print("Fold Accuracies:")  
  
print(scores)  
  
print("\nAverage Accuracy:",  
  
      np.mean(scores))
```

Observation:

Stratified K-Fold maintains approximately the same class distribution in each fold.

Question 39: Train a Naïve Bayes Classifier Using Different Prior Probabilities and Compare Performance.

Python Code:

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score


# Load dataset

data = load_wine()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,
```

```
        test_size=0.2,

        random_state=42,

        stratify=y

    )

# Default priors

model_default = GaussianNB()

model_default.fit(

    X_train,

    y_train

)

pred_default = model_default.predict(

    X_test

)

accuracy_default = accuracy_score(

    y_test,
```

```
        pred_default

    )

    # Equal priors

    model_equal = GaussianNB(

        priors=[1/3, 1/3, 1/3]

    )

    model_equal.fit(

        X_train,

        y_train

    )

    pred_equal = model_equal.predict(

        X_test

    )

    accuracy_equal = accuracy_score(

        y_test,
```



```
        pred_equal

    )

    # Custom priors

    model_custom = GaussianNB(

        priors=[0.6, 0.2, 0.2]

    )

    model_custom.fit(

        X_train,

        y_train

    )

    pred_custom = model_custom.predict(

        X_test

    )

    accuracy_custom = accuracy_score(

        y_test,
```

```
        pred_custom

    )

    print("Default Prior Accuracy:",

          accuracy_default)

    print("Equal Prior Accuracy:",

          accuracy_equal)

    print("Custom Prior Accuracy:",

          accuracy_custom)
```

Observation:

Changing prior probabilities changes the class probabilities used by the classifier. Priors should normally reflect reliable domain knowledge or the intended population distribution.

Question 40: Perform Recursive Feature Elimination (RFE) Before Training an SVM Classifier.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.feature_selection import RFE

from sklearn.pipeline import Pipeline

from sklearn.metrics import accuracy_score


# Load dataset

data = load_breast_cancer()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,
```

```
        test_size=0.2,

        random_state=42,

        stratify=y

    )


# Scale

scaler = StandardScaler()


X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# SVM estimator

estimator = SVC(

    kernel="linear"

)


# RFE

selector = RFE(

    estimator=estimator,
```

```
        n_features_to_select=10

    )

    selector.fit(

        X_train_scaled,

        y_train

    )

    # Transform data

    X_train_selected = selector.transform(

        X_train_scaled

    )

    X_test_selected = selector.transform(

        X_test_scaled

    )

    # Train final SVM

    model = SVC(
```

```
        kernel="linear"

    )

model.fit(

    X_train_selected,

    y_train

)

# Predict

predictions = model.predict(

    X_test_selected

)

accuracy = accuracy_score(

    y_test,

    predictions

)

print("Accuracy After RFE:", accuracy)
```

```
print("Selected Features:")

for feature, selected in zip(

    data.feature_names,

    selector.support_

):

    if selected:

        print(feature)
```

Observation:

RFE recursively removes less useful features and can reduce model complexity.

Question 41: Train an SVM and Evaluate Using Precision, Recall, and F1-Score.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import (

    precision_score,

    recall_score,

    f1_score

)


# Load dataset

data = load_breast_cancer()


X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.2,

    random_state=42,
```



```
        stratify=y

    )

# Scale

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

# SVM

model = SVC(kernel="rbf")

model.fit(

    X_train_scaled,

    y_train

)

# Predict

y_pred = model.predict(
```

```
        X_test_scaled

    )

# Metrics

precision = precision_score(

    y_test,

    y_pred

)

recall = recall_score(

    y_test,

    y_pred

)

f1 = f1_score(

    y_test,

    y_pred

)
```

```
print("Precision:", precision)
```

```
print("Recall:", recall)
```

```
print("F1-Score:", f1)
```

Definitions:

Precision: Of all predicted positives, how many were actually positive.

Recall: Of all actual positives, how many were correctly identified.

F1-Score: Harmonic mean of precision and recall.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Question 42: Train a Naïve Bayes Classifier and Evaluate Using Log Loss.

Python Code:

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import log_loss
```

```
# Load dataset
```

```
data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Gaussian Naive Bayes
```

```
model = GaussianNB()
```

```
model.fit(
```

```
        X_train,

        y_train

    )


# Predict probabilities

probabilities = model.predict_proba(

    X_test

)


# Log Loss

loss = log_loss(

    y_test,

    probabilities

)


print("Log Loss:", loss)
```

Interpretation:

Lower Log Loss is better.

Log Loss evaluates not only whether the predicted class is correct but also how confident the model was in its prediction.

Question 43: Train an SVM Classifier and Visualize the Confusion Matrix Using Seaborn.

Python Code:

```
import seaborn as sns

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import confusion_matrix


# Load dataset

data = load_iris()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Scale
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# SVM
```

```
model = SVC(kernel="rbf")
```

```
model.fit(  
  
    X_train_scaled,  
  
    y_train  
  
)  
  
# Predict  
  
y_pred = model.predict(  
  
    X_test_scaled  
  
)  
  
# Confusion Matrix  
  
cm = confusion_matrix(  
  
    y_test,  
  
    y_pred  
  
)  
  
# Visualize  
  
plt.figure(figsize=(7, 5))
```



```
sns.heatmap(  
  
    cm,  
  
    annot=True,  
  
    fmt="d",  
  
    cmap="Blues",  
  
    xticklabels=data.target_names,  
  
    yticklabels=data.target_names  
  
)  
  
plt.xlabel("Predicted")  
  
plt.ylabel("Actual")  
  
plt.title("SVM Confusion Matrix")  
  
plt.show()
```

Observation:

A confusion matrix shows the number of correct and incorrect predictions for each class.

Question 44: Train an SVM Regressor and Evaluate Using Mean Absolute Error (MAE).

Python Code:

```
from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVR

from sklearn.metrics import mean_absolute_error


# Load housing data

data = fetch_california_housing()

X = data.data

y = data.target


# Split

X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y,

test_size=0.2,

random_state=42

)


# Scale

scaler = StandardScaler()


X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)


# SVR

model = SVR(

    kernel="rbf",

    C=100

)


model.fit(

    X_train_scaled,
```

```
        y_train
    )

    # Predict

    y_pred = model.predict(

        X_test_scaled

    )

    # MAE

    mae = mean_absolute_error(

        y_test,

        y_pred

    )

    print("Mean Absolute Error:", mae)
```

Formula:

MAE=

1

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Observation:

Lower MAE indicates better regression performance.

Question 45: Train a Naïve Bayes Classifier and Evaluate Using ROC-AUC Score.

Python Code:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import roc_auc_score

# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Gaussian Naive Bayes
```

```
model = GaussianNB()
```

```
model.fit(
```

```
    X_train,
```

```
    y_train
```

```
)

# Positive-class probabilities

y_probability = model.predict_proba(

    X_test

)[: , 1]

# ROC-AUC

auc = roc_auc_score(

    y_test,

    y_probability

)

print("ROC-AUC Score:", auc)
```

Interpretation:

ROC-AUC measures how well the model ranks positive examples above negative examples across classification thresholds.

Generally:

- 1.0 → Excellent discrimination
 - 0.5 → Random-level discrimination
 - Below 0.5 → Worse than random ranking
-

Question 46: Train an SVM Classifier and Visualize the Precision-Recall Curve.

Python Code:

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import (

    precision_recall_curve,

    average_precision_score

)

# Load dataset
```



```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
# Scale
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# SVM with probability estimates

model = SVC(

    kernel="rbf",

    probability=True

)

model.fit(

    X_train_scaled,

    y_train

)

# Positive class probability

y_scores = model.predict_proba(

    X_test_scaled

)[: , 1]

# Precision-Recall

precision, recall, thresholds = precision_recall_curve(

    y_test,
```

```
        y_scores

    )

# Average Precision

ap = average_precision_score(

    y_test,

    y_scores

)

# Plot

plt.figure(figsize=(8, 6))

plt.plot(

    recall,

    precision,

    label=f"Average Precision = {ap:.3f}"

)

plt.xlabel("Recall")
```

```
plt.ylabel("Precision")

plt.title("SVM Precision-Recall Curve")

plt.legend()


plt.grid()

plt.show()
```

Observation:

The Precision-Recall curve is particularly useful when the positive class is relatively rare or when precision and recall are more important than overall accuracy.

FINAL REVISION TABLE

Topic	Important Point
SVM	Finds a maximum-margin decision boundary
Support Vector	Important point closest to boundary
Hard Margin	Does not allow training violations
Soft Margin	Allows some violations
C	Controls penalty for classification errors
Gamma	Controls influence of individual points in RBF

Kernel Trick	Helps model non-linear relationships
Linear Kernel	Linear boundary
Polynomial Kernel	Polynomial non-linear boundary
RBF Kernel	Flexible non-linear boundary
SVC	SVM for classification
SVR	SVM for regression
Naïve Bayes	Probability-based classifier
Bayes Theorem	Updates probability using evidence
GaussianNB	Continuous numerical features
MultinomialNB	Counts/frequencies, common in text
BernoulliNB	Binary features
Laplace Smoothing	Prevents zero probabilities
GridSearchCV	Hyperparameter tuning
Stratified K-Fold	Preserves class proportions
RFE	Recursive feature selection
Precision	Correct positive predictions / predicted positives
Recall	Correct positives / actual positives
F1 Score	Harmonic mean of precision and recall
ROC-AUC	Ranking/discrimination across thresholds
PR Curve	Precision-recall trade-off

IMPORTANT FORMULAS

SVM Decision Boundary

$$w^T x + b = 0$$

SVM Margin

$$\frac{2}{\|w\|}$$

Bayes' Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Naïve Bayes

$$P(C|X) \propto P(C) \prod_i P(x_i|C)$$

RBF Kernel

$$K(x,$$

$$x, y) = \exp(-\gamma ||x - x' - y' ||^2)$$

F1 Score

$$F1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Mean Absolute Error

$$MAE = \frac{1}{n} \sum_i |y_i - \hat{y}_i|$$

Mean Squared Error

$$MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$$

CONCLUSION

SVM and Naïve Bayes are important supervised Machine Learning algorithms.

SVM is particularly useful when finding a strong decision boundary is important, and its kernels allow it to handle non-linear relationships.

Naïve Bayes is simple, fast, and especially effective for many high-dimensional text classification problems.

The practical exercises demonstrate how to:

- Train SVM classifiers and regressors
- Use different SVM kernels
- Tune SVM hyperparameters
- Apply feature scaling
- Handle imbalanced datasets
- Use Naïve Bayes variants
- Apply feature selection
- Perform cross-validation

- Evaluate models using different metrics
 - Visualize confusion matrices and Precision-Recall curves
-